Integrating Deep Learning in the Design and Analysis of a Steering Knuckle utilizing DoE

Sivashankar Subramaniam
Department of Mechanical Engineering
Shiv Nadar Institution of Eminence
Delhi NCR, India
ss784@snu.edu.in

Aditya Chandrasekar
Department of Mechanical Engineering
Shiv Nadar Institution of Eminence
Delhi NCR, India
ac416@snu.edu.in

Shravan Kumar Bhadoria
Department of Mechanical Engineering
Shiv Nadar Institution of Eminence
Delhi NCR, India
sb614@snu.edu.in

Ramesh Gupta Burela
Department of Civil Engineering
NITTTR
Bhopal, India
rgburela@nitttrbpl.ac.in

Abstract*— In the current study, we are exhibiting an approach towards the application of Deep Learning to a Finite Element study run by a Design of Experiments setup for a steering knuckle. The steering knuckle is a vital component that has a critical impact on the safety, handling and steering characteristics of an automobile due to the various systems it links and the loading conditions experienced. The design of experiments conducted on the FEM problem includes a parametrisation of the geometric design variables, a material property and loading conditions. The output parameters include maximum von Mises stress, maximum displacement and maximum strain. The prepared dataset is generated for implementation via the deep learning model which comprises of Artificial Neural Networks. They are utilized to best fit the physics-based data of the finite element scenario. The result highlights the ability of the deep learning model to learn non-linear relationships, fit a generalized prediction process and make accurate predictions for this case. Furthermore, with the help of the inverse prediction algorithm created, the optimum values of the design variables, which correlate to minimum stress, strain and displacement are predicted.

Keywords*— Deep Learning, Artificial Neural Networks, Steering Knuckle, Finite Element Method, Design of Experiments, Latin Hypercube

I. INTRODUCTION

The overall feeling of safety and comfort that a vehicle brings to its passengers is thought to be incorrectly dependent on the design of the chassis by people in the community. The steering knuckle, a key component that connects the wheel to the steering and suspension systems, is one of the most important parts of the chassis and has a critical effect on the handling and steering characteristics of the vehicle. In contrast to its size, it has considerable importance. Minimizing the unsprung mass by optimizing and refining components present is a key area of development. This domain has already received attention from the automotive industry in recent years as, to improve a vehicle's handling. Therefore, design optimization of components such as the steering knuckle becomes more prominent.

The intention of this study is to present an approach and an application to develop machine learning competence in the world of computer-aided engineering simulations based on real-world scenarios and, therefore, applications in the real world. Integrating the computational fields of their respective domains and building a framework to train, learn, and predict output metrics of finite element simulations such as equivalent von Mises stress, strain, and deformation, even up to the elemental level utilizing parametric studies and design of experiment setups would be a potent tool to optimize the process of reaching final product designs. Various types of customized Machine learning models would be used to quickly predict the output results of finite element analysis on the basis of training data without performing the CAE analysis, thereby accelerating product design and aiding in predicting computationally and numerically intensive problem statements. FEA simulations were performed on ABAQUS CAE. Simulia Isight has been used to formulate and design experiments to automate and generate the CAE analysis process.

Our review of the existing literature suggests that research and development in engineering design while integrating machine learning and artificial intelligence tools demonstrates the way forward to design optimization.

Soyoung Yoo et al. (2021) [1] worked on integrating artificial intelligence into computer-aided design and computer-aided engineering by developing a deep learning-based framework. The work proposed a method to produce 3D designs and evaluate their structural performance. Wen-Ren Jong et al. (2020) [2] pointed towards the implementation of the Taguchi method and the incorporation of artificial neural networks to explore machine learning in CAE. Despite recent growth in CAE, limited by lengthy analysis times, it falls short for on-site real-time assessments. The Back Propagation Neural Network (BPNN), known for its predictive strength in nonlinear problems, offers a swift and accurate alternative. In this study, Jong et al. trained BPNN using CAE analysis data and optimized hyperparameters using the Taguchi orthogonal method, resulting in a predictive neural network for CAE analysis. Christopher P. Kohar et al (2021) [3] proposed a novel framework to applying machine learning models to FEA simulations of dynamic axial crushing of tubes. They used explicit design parameters such as the wall's thickness and size and meshing techniques to prepare the training data.

Following this, we examine the contributions of various authors, with special attention to the insights provided by our research work. With regards to an industry application standpoint, in their literature review, Brunton et al. (2021) [4] explored the integration of data-driven science and engineering in the aerospace industry, emphasizing the need for interpretable and certifiable machine learning techniques. The review included a retrospective analysis, an assessment of the state-of-the-art, and a forward-looking roadmap. Sakaridis et al. (2022) [5] proposed a machine-learning methodology for predicting the buckling response of tubular structures. They generated a dataset of force-time curves using a calibrated

finite element model within a parametric space with a highly non-linear buckling response. Utilizing a fully connected neural network template, Sakaridis determined machine learning hyper-parameters and evaluated the resulting model on a separate test set, focusing on maximum and average load and energy absorption errors. Jice Zeng et al. (2023) [6] published work on improving the prediction accuracy of the CAE model by fusing CAE data and a small sample set of crash test data. This showed the discrepancies that can exist and how they can be overcome with regards to correlation issues between computational and practical results.

With regards to the finite element analysis formulation of the steering knuckle, the loads on the steering knuckle connection points are derived on the basis of various driving manoeuvres, namely acceleration, braking, bump load and cornering. The load distribution on the basis of g-forces, as exhibited by Abdullah et al. (2011) [7], is used as a reference for the CAE model.

Shelar et al. [8] addressed a crucial issue in the vehicle industry concerning variations in physical properties and manufacturing methodologies. Deterministic approaches fall short in accommodating these variabilities, necessitating a robust methodology based on strength and design optimization through probabilistic models of design variables (DOE). Focusing on the steering knuckle, a critical vehicle component, they optimized the design using a methodology grounded in durability and probabilistic models of design variables. Park et al. [9] utilized a DoE to optimize the design of an aluminium alloy knuckle in a suspension system. They employed an orthogonal array strategy during structural optimization, treating relevant discrete variables as levels. Since the conventional orthogonal array did not consider the constraint, a characteristic function was introduced to consider constraint feasibility. This innovation expanded the applicability of general DOE to problems involving constraints, enhancing the optimization process.

The current study attempts to extend existing research by taking forward the application of machine learning on a critical automotive component. The training data for the framework is finite element simulations on a parametrised steering knuckle. A dataset comprising 6000 unique design points was prepared via a Latin hypercube DoE by parametrizing the three different types of variables: (I) geometric design variables, (II) material design variables (III) loading conditions. This dataset serves as the deep learning model's training data, which can predict the output stress, strain and displacement value for a given design and loading scenario. Another novel output of the model is to predict the optimal combination of design parameters for minimum stress, strain and displacement given a loading case. We refer to this as the Inverse Propagative Approach. Validated examples are given to demonstrate the efficiency and legitimacy of the proposed process. These results indicate that the proposed approach not only reduces the computational cost dramatically but also guarantees convergence.

II. METHODOLOGY

A. *Finite Element Analysis Formulation of Steering Knuckle*

In the dynamic scenario of a moving vehicle, the steering knuckle connection points undergo diverse forces. Four primary load conditions—acceleration, braking, bump, and cornering—are under consideration. It's crucial to acknowledge that the forces acting on each knuckle differ due to the variable centre of gravity. For this specific case, we'll examine a vehicle executing a left turn with simultaneous braking and encountering a bump. The Front-Left Knuckle is the focal point for all load cases. The research conducted to determine these loads also accounts for dynamic weight transfer during various phases of the vehicle's motion, such as braking. This consideration of varying centre of gravity plays a significant role in calculating the loads on the steering knuckle.

When a vehicle accelerates, the centre of gravity experiences an inertia force which results in a weight transfer from the front to the rear. Considering a gravity force 0.5g and the forces acting on the vehicle along the wheelbase, hence reaction forces will be generated on both wheels. During the process of braking, an inertia force comes into play, which affects the vehicle's centre of gravity. This force leads to a transfer of mass from the rear of the vehicle towards the front. This translated to an impact force of 1.1G. The equilibrium equation of the loads on the front wheel is mentioned in Eq. (1)

$$W_L = \frac{1}{2}mg\frac{a_1}{L} - \frac{1}{2}mg\frac{h_2}{L}\frac{a}{g} \qquad (1)$$

When a vehicle goes over a bump, according to Smith (1978), it causes the most amount of stress to develop. The lower suspension arm mounting point suffers a load of approximately 2.5G. The upper suspension arm mounting point suffers a load of approximately 1.1G. It can also be noted that bump scenario has no effect on the centre of gravity of the vehicle. When taking a turn, the car shifts its weight from the outer wheel to the inner wheel as it follows a specific radius of curvature around a centre. This causes the load to transfer in the lateral direction, where, L is the track width, W_L is the load on front left tire, a is the acceleration of the vehicle (1.2g), m is the mass of the vehicle, a_1 is the distance between the CoG under dyanmic acceleration and the centre of the front wheel when viewed from the side, a_2 is the distance between the CoG under dynamic acceleration and the centre of the rear wheel when viewed from the side and h_2 is the distance between the floor to the CoG under dynamic acceleration. The equilibrium equation during cornering is mentioned in Eq. (2)

$$m \times g \times a_2 + C.G_{.height} \times m \times a = W_L \times L \qquad (2)$$

Since loads for only one knuckle are to be calculated, the load distribution for this case will be different hence, the standard equation of 1G will change to:$1G = \frac{0.35}{4}mg$. Figure 1 displays the load distribution between the mounting points of the steering knuckle in terms of 'g' forces. Table 1 exhibits the derived load value distributed between the mounting points of the steering knuckle.

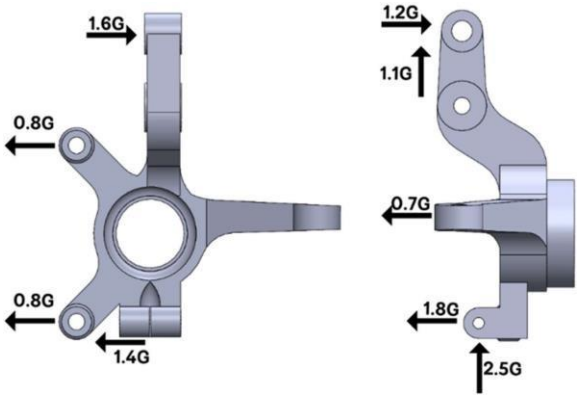

Fig. 1. Load components and their application on the steering knuckle.

TABLE I. DISTRIBUTED LOAD CALCULATIONS

Force	***G* Load**	***MG***	**Load Value (1*G*)**	**Distributed Load Value (0.35 X 0.25 *G*)**
F1	1.2	10594.8	12713.76	1112.45
F2	1.1	10594.8	11654.28	1019.74
F3	0.7	10594.8	7416.36	648.93
F4	1.8	10594.8	19070.64	1668.68
F5	2.5	10594.8	26487	2317.61
F6	1.6	10594.8	16951.68	1483.27
F7	0.8	10594.8	8475.84	741.63
F8	1.4	10594.8	14832.72	1297.86

The central hub is constrained to simulate the scenario wherein the wheel hub is fixed to the knuckle with the help of a bearing. The loads are to be applied as surface tractions for the Finite Element Analysis setup on Abaqus CAE. Having calculated the load in Newton terms, we will need to find the surface area of each steering knuckle mounting point to calculate the required surface traction.

Upper Suspension Mounting (A) - *1272.35 mm*^{2}
Brake Caliper Mounting (B) - *1315.7 mm*^{2}
Lower Suspension Mounting (C) - *1014.71 mm*^{2}
Tie Rod Mounting Arm (D) - *629.81 mm*^{2}.
The calculated surface traction loads are listed in Table 2.

TABLE II. SURFACE TRACTIONS INPUTS

Force	Distributed Load value for Front Left Knuckle (0.35 × 0.25) (N)	Surface Traction Input Magnitude (MPa)
X1	1112.4540	0.8740
X2	1019.7495	0.8010
X3	648.9315	1.0303
X4	1668.6810	1.6444
X5	2317.6125	2.2840
X6	1483.2720	1.1650
X7	741.6360	0.5630
X8	1297.8630	1.2790

The Abaqus CAE-based finite element (FE) model uses a 5mm mesh size to ensure converged results. The CAE file and Output Database (ODB) file generated from the simulation are required to enable the CAE Automation process which we would later achieve using Isight. Insight would be used to run the Abaqus solver in a defined approach, conducting parametric studies over the initially generated CAE and ODB files, enabling efficient parametric design optimisation.

B. Parametrisation of the design variables

The success of design optimization and parametric exploration hinges on selecting pivotal design zones that offer scope for improvement individually as well as in combined effect with other such design areas. These may be regions where there is a prevalence of increased stress concentration or excess material. It is noticeable from initial FEA studies that the arm extending from the hub to towards the upper suspension strut and the lower tie suspension mounting could be required to bear significant sudden loads and, thereby, stresses. It is also noted that the tie rod arm is a recipient of considerable displacement. On the basis of observations, six geometric design variables have been selected to be parametrized, as shown in Fig. 2.

The fillet radius between the lower suspension arm and the upper suspension arm has been considered as a parameter, namely R1 and R2. The fillet radius at the junction between the tie rod arm and the hub is parametrized as R3. The fillet radius at the junction between the hub and the upper suspension strut mounting point is referred to as design variable R4. Another important design variable is R5, which is the oblique junction between the upper suspension strut mounting and the hub. The thickness of the tie rod mounting arm is parametrized as well: L1 with the intention of decreasing the deflections arising.

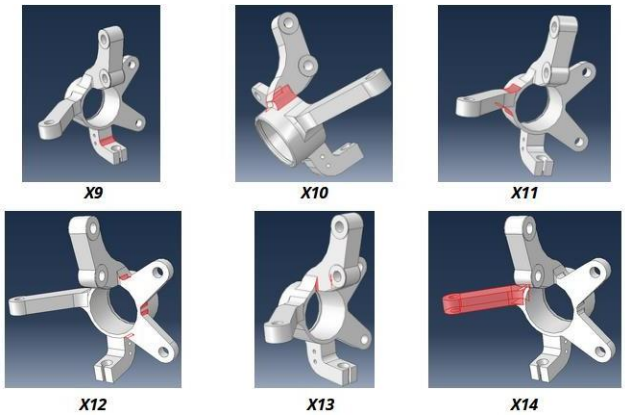

Fig. 2. Illustration of various geometric design parameters

The eight load variables and the material property of Young's modulus will be parameterized to generate robust training data. This enables us to analyse how different factors impact system behaviour. Young's modulus, a measure of material stiffness, will be included to understand how materials deform or resist deformation under applied forces. This is key to predicting performance and ensuring the reliability of components. Parametrizing these elements helps us create a dataset that reflects a variety of scenarios. This approach allows us to simulate real-world conditions and evaluate performance across a spectrum of configurations. It leads to more reliable predictive models and informs design decisions to improve system reliability and safety.

C. Design of Experiments (DoE)

The success of Machine Learning (ML) and Deep Learning (DL) models hinges on the careful preparation of a robust and meaningful dataset. This crucial preparation phase significantly influences the model's capability, accuracy, and overall performance. A design of experiments will be defined and run to have a structured approach towards the parametrization of variables and the versatility of the datasheet that would be generated for ML model implementation. The methodical and numerical approach of the Design of Experiments (DoE) can be utilized in finite element analysis for structural optimization. In this procedure, we choose and control particular design variables, such as geometric factors, to investigate their impact on structural efficacy via running a series of simulations wherein modifications are implemented to a few of the input design variables, and the correlation between multiple input variables (Factors) and key output variables (Responses) is investigated.

1) Definition of design variables

Recognizing control factors is an essential stage, as they are the main variables that are deliberately altered during the investigation to examine their effect on the dependent variable. We have control over these independent variables and can adjust at specific levels to investigate their influence on the system's behaviour.

2) Full Factorial Method

The initial approach to DoE employed using a full factorial method to generate the training data. The full factorial method is a systematic approach used in experimental design to explore all possible combinations of input variables or factors at various levels. In our case, where there are 8 load parameters, 6 geometric parameters, and 1 material property, the full factorial method would entail considering every possible combination of these parameters within their specified ranges. The factors include load parameters, geometric parameters, and material property (Young's modulus). Each factor has multiple levels or values that it can take within its specified range. Each combination of factors represents a unique scenario for the steering knuckle.

However, the full factorial method was realized to be computationally expensive and time-consuming because of operating with a large number of factors and levels. In our case, with multiple parameters to consider, the full factorial method required a very large number of simulations, leading to significant computational resources and time requirements.

3) Latin Hypercube Method

The Latin Hypercube Sampling (LHS) proved to be advantageous for this particular use case of ours. LHS provides a more efficient sampling strategy by selecting a subset of representative points from the entire parameter space, thereby reducing the number of simulations needed while still maintaining a diverse set of combinations. LHS ensures a good spread of data points within the defined range for each variable. This allows for capturing the entire design space with a significantly lower number of simulations compared to a full factorial design. This reduction in simulations translates to shorter computational times and resource savings. Unlike a full factorial design, LHS avoids clustering data points in specific regions of the design space. It guarantees at least one data point in each interval for each variable, leading to a more uniform exploration of the possible solutions. This characteristic is particularly beneficial for neural network training, as it helps the model learn the relationship between input variables and outputs across the entire design space. Based on these advantages, LHS was determined to be a more efficient and effective DoE method for generating a dataset suitable for training and utilizing the neural network model.

D. Usage of Simulia Isight

The software suite employed in this research consisted of two primary tools namely Isight and Abaqus. Simulia Isight acts as a central design optimization platform. It allows for defining the design variables (loads, geometry, and material properties), specifying the DoE method (in this case, LHS), and interfacing with external analysis software which was Abaqus. Abaqus is a finite element analysis (FEA) software that performs the structural analysis of the steering knuckle for each design iteration generated by Isight. Abaqus takes the input values from Isight for the 15 features (8 loads, 6 geometric parameters, and 1 material property) that are constantly parametrized according to the LHS DoE method. It then simulates the structural behaviour of the knuckle under the specified loads and calculates the corresponding stress, strain, and displacement values.

We have used SIMULIA Isight not only for the automation of the Abaqus Solver but also to integrate the Design of Experiment Process component seamlessly. This integration ensured that the data generated by automated simulations directly aligned with the factor and level definitions established in the Design of Experiments setup. Fig. 3 and Fig. 4 display the setup procedure in the design gateway and the Simflow in the runtime gateway as displayed in the Isight GUI respectively. In essence, Isight manages the workflow by sending the design variations to Abaqus for analysis while Abaqus returns the simulation results (stress, strain, and displacement) back to Isight. This process iterates for all the design points identified by the LHS method, resulting in a comprehensive dataset that captures the relationship between the design variables and the structural response of the steering knuckle.

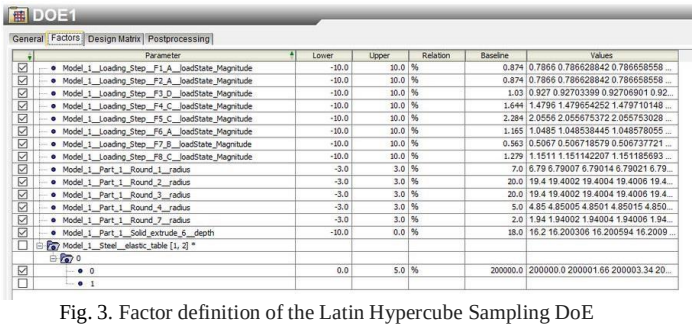

Fig. 3. Factor definition of the Latin Hypercube Sampling DoE

This approach marked a departure from conventional methods, as it eliminated the reliance on a Python code running a for loop of values to be executed via a sheet or .inp file input. The utilization of SIMULIA Isight introduced a more sophisticated and streamlined mechanism for executing simulations in accordance with the designated experimental design, thereby enhancing the efficiency and precision of the data generation process.

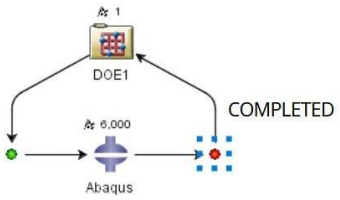

Fig. 4. Simflow Setup on Simulia Isight

E. Machine Learning Algorithm: Neural Networks and Inverse Propagation

An artificial neural network (ANN) is a computational model comprised of interconnected artificial neurons. ANNs mimic the learning and processing capabilities of the human brain. The ANN consists of three main components: the input layer, hidden layer, and output layer. These layers are interconnected through synapses, which allow the transfer of signals or information. They are widely used for tasks such as data fitting and pattern recognition. Once trained with the relevant dataset, an ANN can be used to make predictions or estimate values for new independent data points. In the present study, a dataset that comprises 6000 rows was generated using the Isight and Abaqus software by varying the 15 parameters, out of which 8 were load parameters, 7 were geometric parameters, and the other was the Young's Modulus as discussed above.

1) Neural Network Mechanism

The neural network algorithm operates through a series of interconnected layers of neurons, each performing specific computations to transform input data into meaningful output. At the core of this algorithm lies the concept of weighted sum, where each input is multiplied by a corresponding weight, and these products are summed together along with a bias term. This aggregated value is then passed through an activation function, which introduces non-linearity into the system by determining whether the neuron should be activated based on its input.

Once the output is generated, the algorithm evaluates its performance using a predefined loss function, which measures the disparity between the predicted output and the actual target. This discrepancy serves as a guide for adjusting the weights and biases in the network to minimize the loss and enhance predictive accuracy.

This adjustment process, known as backpropagation, involves calculating the gradient of the loss function with respect to each weight and bias in the network. This gradient represents the direction and magnitude of change required to minimize the loss. Utilizing the chain rule of calculus, the algorithm propagates this gradient backward through the network, updating the weights and biases of each neuron accordingly.

Gradient descent is the optimization technique employed during backpropagation to iteratively adjust the weights and biases in the direction opposite to the gradient, thereby descending along the steepest slope of the loss function. By iteratively repeating this process over multiple epochs, the algorithm gradually converges towards a set of optimal weights and biases that minimize the loss and maximize the network's predictive performance. The following explains the individual process components in further detail.

a) Weighted Sum

The first step in the neural computation process involves aggregating the inputs to a neuron, each multiplied by their respective weights, and then adding a bias term. This weighted sum combination is expressed as: $z = \sum_{i=1}^{n} w_i x_i + b$ where: z is the weighted sum, w_i represents the weight associated with the i-th input, x_i is the i^{th} input to the neuron, b is the bias term, a unique parameter that allows adjusting the output along with the weighted sum. Fig. 5. shows the mathematical formulations of a neuron.

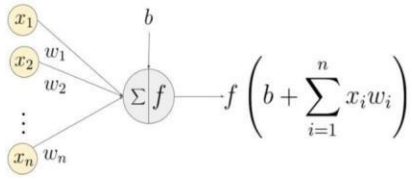

Fig. 5. Mathematical Formulation of a Neuron

b) Activation Functions

Activation functions serve to introduce non-linearity into the neural network, enabling it to learn complex patterns and relationships within the data. They are essentially mathematical operations applied to the output of a neuron in a neural network. They determine whether the neuron should be activated or not based on the weighted sum of its inputs. In a formal sense, activation functions receive a weighted sum of inputs from the previous layer of neurons, denoted as $z = \sum_{i=1}^{n} w_i x_i + b$ as mentioned above.

The activation function then applies a non-linear transformation to this weighted sum, producing the output of the neuron, denoted as $a = f(z)$ where f is the activation function. This output is then passed on to the next layer of neurons as inputs. The following are some of the most commonly used activation functions.

The Sigmoid function has an S-shaped curve and maps any real number to a value between 0 and 1. This characteristic makes it suitable for binary classification tasks, where the output can be interpreted as the probability of an instance belonging to a particular class. It is given as $f(x) = \frac{1}{1+e^{-x}}$. Fig 6. shows the plot of the sigmoid function. The ReLU activation function outputs the input itself if it's positive, and zero otherwise. This piecewise-linear function is computationally efficient. It helps mitigate the vanishing gradient problem by not saturating in the positive region, which accelerates the convergence of gradient-based optimization algorithms. It is given by f(x)=max (0, x). Fig 7. shows the plot of the ReLU function.

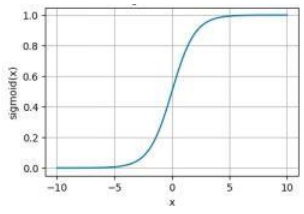

Fig. 6. Sigmoid Function

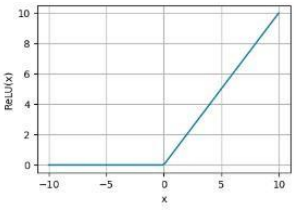

Fig. 7. ReLU Function

The softmax function plays a crucial role in the output layer of multi-class classification problems. It transforms an array of real numbers into a probability distribution. Each element in the output vector represents the probability of an input belonging to a specific class. The softmax function

ensures all outputs lie between 0 and 1 and sum to 1, satisfying the properties of a valid probability distribution. It is given by, $f(x)_i = \frac{e^{x_i}}{\sum_j e^{x_j}}$, where the input to the activation function is a vector of real numbers denoted as $x_0, x_1, \ldots, x_n$. Fig 8. shows the plot of the softmax function.

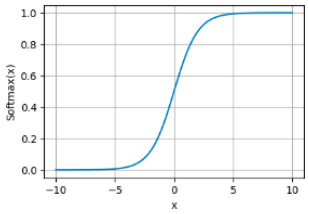

Fig. 8. Softmax Function

c) Backpropagation

Backpropagation is a method for calculating the gradient of the loss function concerning all weights in the network. It consists of a backward pass, where the output is compared to the target value, and the error is propagated back through the network to update the weights. The chain rule for backpropagation can be represented as: $\frac{\partial L}{\partial w} = \frac{\partial L}{\partial a} \frac{\partial a}{\partial z} \frac{\partial z}{\partial w}$ where $\partial a/\partial L$ is the gradient of the loss function to the activation, $\partial z/\partial a$ is the gradient of the activation function to the weighted input z, $\partial w/\partial z$ is the gradient of the weighted input to the weight w, z represents the weighted sum of inputs and a is the activation.

d) Gradient Descent

Gradient Descent is an optimization algorithm used for minimizing the loss function in a neural network. The loss function utilised here is the Mean Squared Error (MSE) $= \frac{1}{n}\sum_{i=1}^{n} (Y_i - \hat{Y}_i)^2$ where n is the number of data points, Y_i are the observed values and $\hat{Y}_i$ are the predicted values. The gradient descent algorithm works by iteratively moving the weights in the direction of the steepest decrease in loss. The amount by which the weights are adjusted in each iteration is determined by the learning rate, a hyperparameter that controls the size of the steps. The weight update rule in gradient descent can be expressed as: $w_{\text{new}} = w_{\text{old}} - \eta \cdot \frac{\partial L}{\partial w}$ where w_{new} and w_{old} represent the updated (new) and current (old) values of the weight, respectively, η is the learning rate, a hyperparameter that controls the size of the step taken in the direction of the negative gradient, $\partial w/\partial L$ is the gradient of the loss function for the weight.

F. Neural Network Model Implementation

The research utilized a combination of Python libraries for data analysis and machine learning. Pandas facilitated data loading, preprocessing, and exploration. TensorFlow served as the foundation for building and training the neural network models, including defining the architecture, loss function, and optimization algorithm. Scikit-learn provided tools for data preprocessing tasks like feature scaling. These libraries empowered the research to effectively process data and develop a robust neural network model for the design and analysis of the steering knuckle.

1) Model Architecture

Input Layer: The first layer received the pre-processed input vector containing the 15 design variables (8 loads, 6 geometric parameters, and 1 material property).

Hidden Layers: The model utilized multiple hidden layers with a varying number of neurons in the manner of 64, 32 and 16 to capture the complex relationships between the input features and the target variable. ReLU activation functions were employed in these layers to introduce non-linearity and improve model performance.

Output Layer: The final layer had a single neuron which predicts the continuous target value (stress, strain, or displacement).

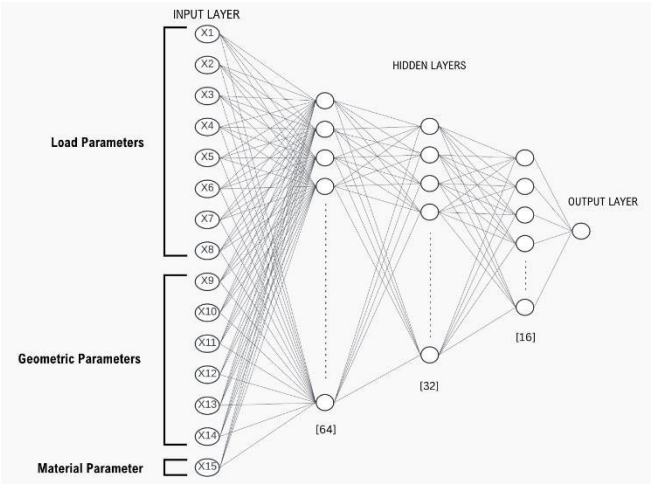

Fig. 9. Chosen Architecture of the Neural Network Model

2) Training and validation process

The training process involved fitting the separate neural network models to the pre-processed data. Adam (Adaptive Moment Estimation), an efficient optimization algorithm, was used to minimize the mean squared error (MSE) loss function between the predicted and actual target values. Further, to prevent overfitting, a common challenge in machine learning, the training employed an Early Stopping callback. This callback monitored the validation loss during training and terminated the process if the validation loss did not improve for a predefined number of epochs. The validation process involved comparing the predicted values by the neural network of the target variables, which are the stress, strain and displacement, to the actual values, which are calculated by the Abaqus CAE software. Three test cases where we define the values of the 15 parameters were examined to calculate the accuracy of the neural network model, which is explained in detail in section 3.

G. Inverse Prediction Algorithm

This work introduces the Inverse Prediction algorithm that uses the trained model's predictive abilities to identify the steering knuckle's optimal geometric properties that minimize stress, strain, and displacement in a structure when given the load conditions. The approach leverages a pre-trained neural network model, initially created to predict these outputs given a set of features. These features encompass various aspects of the structure, including the geometric parameters. The inverse prediction algorithm iteratively modifies the feature set within the pre-trained model. This approach creates and utilizes a full factorial design with refined levels for each factor. This refined design creates a significantly more detailed dataset than the one used to train the original model. By systematically evaluating each combination of features within this expanded dataset using the pre-trained model, the algorithm independently identifies

the specific combination that leads to the minimum values of stress, strain, and displacement.

In essence, the inverse propagative approach utilizes the predictive power of the pre-trained model to navigate a high-dimensional space of potential geometric configurations. This allows for the efficient identification of the optimal combination that minimizes the undesired outputs of stress, strain, and displacement.

Given below is a summary of the approach taken:

- Identified and defined loads acting on the steering knuckle. Identified 8 load parameters, 6 geometric parameters, and 1 material property. Young's modulus was the sole material property considered.
- Isight software was used for dataset generation and set to create a dataset that followed the Latin Hypercube DOE technique. Isight was set to govern the Abaqus software, which was used for structural analysis and produce the dataset.
- Created separate neural network models for predicting stress, strain, and displacement.
- Fitted models using Adam optimization and Early Stopping callback to prevent overfitting.
- Chose three test cases and compared predicted values to actual values from Abaqus.
- Utilized the Inverse Prediction algorithm which uses the trained neural network to identify optimal geometric properties of steering knuckle to minimize stress, strain, and displacement given load conditions.

III. Results and Discussions

A validation study was conducted to evaluate the performance of the neural network model that was built. The study involved taking three samples of possible values that the 15 parameters aforementioned can take and then passing them on to the neural network model for it to predict the corresponding values of strain, stress and displacement.

The following test cases which were fed into the neural network model. The results were further validated and compared to the actual value for the same inputs in ABAQUS. The results are listed down in Table 3, Table 2, Table 3 and Table 4.

TABLE III. TEST CASES

Parameters	Variable Tag	Test case 1	Test case 2	Test case 3
Load 1 (MPa)	X1	0.791	0.850	0.786
Load 2 (MPa)	X2	0.931	0.910	0.863
Load 3(MPa)	X3	1.222	1.100	1.001
Load 4(MPa)	X4	1.441	1.610	1.680
Load 5(MPa)	X5	2.221	2.080	2.264
Load 6 (MPa)	X6	1.221	1.100	1.193
Load 7 (MPa)	X7	0.672	0.530	0.563
Load 8 (MPa)	X8	1.206	1.340	1.298
Geometric 1 (mm)	X9	7.000	7.200	6.969
Geometric 2 (mm)	X10	19.000	20.200	19.446
Geometric 3 (mm)	X11	19.000	20.000	20.207
Geometric 4 (mm)	X12	5.000	5.120	5.145
Geometric 5 (mm)	X13	2.000	1.980	2.034
Geometric 6 (mm)	X14	17.000	17.000	17.967
Young's Modulus (MPa)	X15	200000	200000	202140

Table IV. Performance Table of the ANN

S. No.	Target Variables	Neural Network Model Predictions	Abaqus Results
1	Max Strain	0.00119506	0.00100700
	Max Stress (MPa)	195.66934	191.50000
	Max Displacement (mm)	0.0764452	0.0745300
2	Max Strain	0.0009263	0.0009653
	Max Stress (MPa)	179.30050	184.60000
	Max Displacement(mm)	0.07179974	0.07176000
3	Max Strain	0.0009679	0.0010830
	Max Stress (MPa)	164.69520	168.40000
	Max Displacement (mm)	0.0706400	0.0724900

The accuracy achieved for target variables, namely strain, stress, and displacement, are 96%, 97.86% and 97.45%, respectively. Furthermore, the overall accuracy achieved by the model is 97.103%.

The inverse prediction algorithm now comes into play after a good accuracy for the neural network model is achieved. Table 6 displays the results of the inverse prediction algorithm which predicts the optimal values of the geometric parameters for a specified loading condition. Table 5 shows the loads taken for this case.

Table V. Loading Case defined for the Inverse Prediction Algorithm

Parameter	Value
Load1($X1$)	0.874
Load2($X2$)	0.874
Load3($X3$)	1.03
Load4($X4$)	1.644
Load5($X5$)	2.284
Load6($X6$)	1.165
Load7($X7$)	0.563
Load8($X8$)	1.279

Table VI. Prediction Table of the Inverse Prediction Algorithm

Target Variables	Optimal Values of the Geometric Parameters (mm)					
	X9	X10	X11	X12	X13	X14
Strain	6.6	19.0	19.4	4.8	2.0	16.0
Stress	6.6	19.0	19.6	4.8	2.0	16.4
Displacement	6.6	19.0	19.6	4.8	2.0	16.0

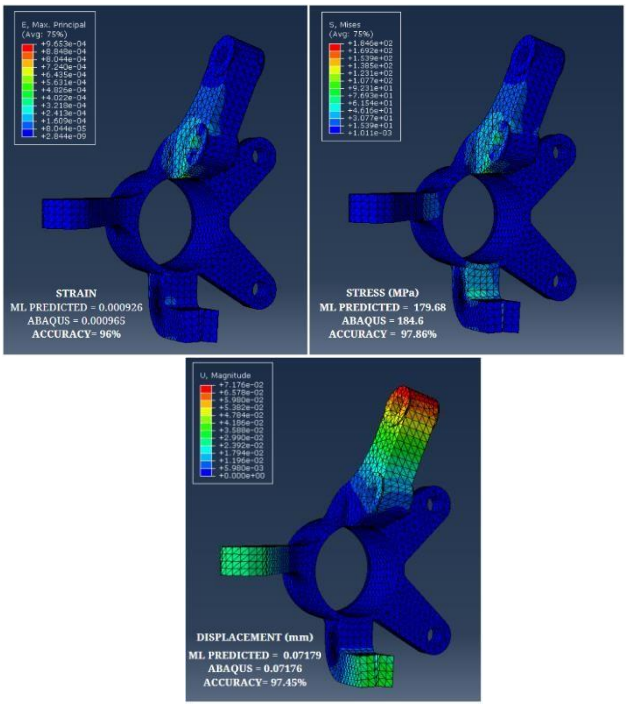

Fig. 10. Abaqus CAE Validation of the test case 2

IV. Conclusion

The current investigation into incorporating artificial neural networks in a steering knuckle's design and analysis process can successfully predict the stress, strain, and displacements experienced for a given test case. Furthermore, with the help of the novel inverse prediction algorithm created, the optimum values of the design variables, which correlate to minimum stress, strain and displacement are predicted. In addition, there has been a significant advantage of using the Latin Hypercube DoE method for our Deep Learning model development as it drastically reduced the number of simulations required while improving factor space representation. It ensured a relatively robust and better distribution sampling points.

This collaboration of ML and CAE can also be further extended to the thermal domain as well as dynamic scenarios. A transient thermal analysis of the brake disc rotor can serve as a test scenario for the thermal scenario, while the vibration of the clamped plate can serve as the test case for a dynamic analysis, which can further be extrapolated to real-life scenarios. This process outlined in the research work so far can also be run against, as well as in combination with other contemporary tools of design optimization, such as generative design and topology optimization for benchmarking studies and further optimization.

References

[1] Yoo, S., Lee, S., Kim, S., Hwang, K. H., Park, J. H., & Kang, N. (2021). Integrating deep learning into CAD/CAE system: generative design and evaluation of 3D conceptual wheel. Structural and multidisciplinary optimization, 64(4), 2725-2747.

[2] Wen-Ren Jong, Yan-Mao Huang, Yun-Zih Lin, Shia-Chung Chen & Yu-Wei Chen (2020): Integrating Taguchi method and artificial neural network to explore machine learning of computer-aided engineering, Journal of the Chinese Institute of Engineers, DOI: 10.1080/02533839.2019.1708804

[3] Christopher P. Kohar, Lars Greve, Tom K. Eller, Daniel S. Connolly, Kaan Inal, A machine learning framework for accelerating the design process using CAE simulations: application to finite element analysis in structural crashworthiness, Computer Methods in Applied Mechanics and Engineering, Volume 385, 2021, 114008, ISSN 0045-7825, https://doi.org/10.1016/j.cma.2021.114008.

[4] Data-Driven Aerospace Engineering: Reframing the Industry with Machine Learning Steven L. Brunton, J. Nathan Kutz, Krithika Manohar, Aleksandr Y. Aravkin, Kristi Morgansen, Jennifer Klemisch, Nicholas Goebel, James Buttrick, Jeffrey Poskin, Adriana W. Blom-Schieber, Thomas Hogan, and Darren McDonald

[5] Emmanouil Sakaridis, Nikolaos Karathanasopoulos, Dirk Mohr, Machine-learning- based prediction of crash response of tubular structures, International Journal of Impact Engineering, Volume 166, 2022, 104240, ISSN 0734-743X

[6] Zeng, J., Li, G., Gao, Z. et al. Machine learning enabled fusion of CAE data and test data for vehicle crashworthiness performance evaluation by analysis. Struct Multidisc Optim 66, 96 (2023). https://doi.org/10.1007/s00158-023-03553-5

[7] Abdullah, S. H., Ullah, M. M., & Sharief, O. (2011). Design and Multi-Axial Load Analysis of Automobile Steering Knuckle. Carbon, 3, T6.

[8] Shelar, M. L., & Khairnar, H. P. (2014). Design analysis and optimization of steering knuckle using numerical methods and design of experiments. International Journal of Engineering Development and Research, 2(3), 2958-2967.

[9] Park, Young & Kang, Jung & Lee, Dong & Oh, Seung & Ko, Won & Lee, Kwon. (2007). Shape Optimization of a Knuckle using the Design of Experiments. Key Engineering Materials - KEY ENG MAT. 345-346. 905-908. 10.4028/www.scientific.net/KEM.345- 346.905.